ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Костюк Ю. Л.

МЕТОДЫ КОМПИЛЯЦИИ

Лекция 2

АВТОМАТНЫЕ ГРАММАТИКИ И ЯЗЫКИ

Класс 3: автоматные грамматики (А-грамматики).

Вид порождающих правил:

$$A \to aB \quad \text{или } A \to a$$

где A, B – нетерминалы, a – терминал.

Пример автоматной грамматики

Грамматика: $G_2(L) = \{\sum, N, S, P\}$

$\sum = \{a, b\}$, $N = \{S, T\}$, $S = \{S\}$,

$P = \{S \to aS, S \to bT, T \to bT, T \to a\}$.

Процесс порождения:

$S \Rightarrow aS \Rightarrow abT \Rightarrow aba$,

$S \Rightarrow bT \Rightarrow bbT \Rightarrow bbbT \Rightarrow bbba$, . . .

Со второго шага порождения цепочка имеет вид:

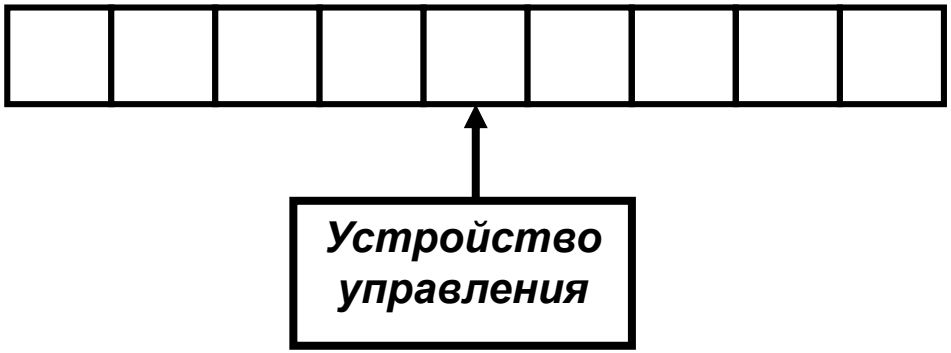
$$\gamma A,$$

где γ – терминальная цепочка, A – нетерминал, на последнем шаге нетерминал A заменяется на терминал.

РАСПОЗНАВАТЕЛЬ АВТОМАТНОЙ ГРАММАТИКИ – КОНЕЧНЫЙ АВТОМАТ

Конечный автомат – это частный вид машины Тьюринга, в которой:

1) лента является входной, на каждом шаге работы автомата с неё считывается очередной символ;
2) на каждом шаге работы автомата устройство управления делает переход в новое состояние в соответствии с правилами перехода.



Конечный автомат задается пятеркой множеств:

$$\{\Sigma, Q, q_0, F, \delta\},$$

где Σ – множество (алфавит) входных символов;

Q – множество состояний автомата;

q_0 – начальное состояние $q_0 \subset Q$;

F – множество заключительных состояний $F \subseteq Q$;

δ – множество правил перехода, имеющих вид:

$$(a, q_i) \rightarrow q_j,$$

Правило перехода задаёт переход из состояния q_i, когда на входе читается символ a, в состояние q_j.

РАСПОЗНАВАТЕЛЬ АВТОМАТНОЙ ГРАММАТИКИ – КОНЕЧНЫЙ АВТОМАТ

Начало работы: считывающая головка автомата обозревает самый левый символ на ленте; автомат находится в начальном состоянии q_0.

На очередном шаге – переход в новое состояние в соответствии с правилом перехода:

$$(a, q_i) \rightarrow q_j,$$

лента продвигается влево на один символ.

Если прочитаны все символы на ленте и автомат находится в заключительном состоянии, то говорят, что входная цепочка успешно распознана (*допущена*) автоматом.

В противном случае - цепочка *не допущена*.

КОНЕЧНЫЙ АВТОМАТ: РЕАЛИЗАЦИЯ

1. Справа после входных символов на ленте автомата – «пустая» клетка (символ «$\perp$»), ***не принадлежащий алфавиту*** $\sum$.
2. Каждому нетерминалу грамматики соответствует одно состояние конечного автомата.
3. В грамматику добавляется еще один нетерминал Z, ему соответствует заключительное состояние. Правила вида: $A \rightarrow a$ ***заменяются на*:** $A \rightarrow aZ$, кроме того, ***добавляется правило*:** $Z \rightarrow \perp$.
4. Успешному распознаванию в работе конечного автомата соответствует следующая ситуация:

- головка автомата обозревает символ «$\perp$»;
- автомат находится в заключительном состоянии.

Тогда входная цепочка ***допущена*** автоматом.

ПРИМЕР КОНЕЧНОГО АВТОМАТА

Грамматика: $G_2(L) = \{\sum, N, S, P\}$

$\sum = \{a, b\}$, $N = \{S, T\}$, $S = \{S\}$,

$P = \{S \to aS, S \to bT, T \to bT, T \to a\}$. Замена $T \to a$ на $T \to aZ$.

Добавление: $Z \to \perp$. Автомат в виде таблицы:

Входной символ / Состояние	a	b	$\perp$
S	S	T	
T	Z	T	
Z			*допуск*

Примеры:

$$\begin{array}{ccccccc} a & a & a & b & b & a & \perp \\ \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ S & S & S & S & T & T & \boldsymbol{Z} \end{array} \qquad \begin{array}{ccc} a & b & \perp \\ \uparrow & \uparrow & \uparrow \\ S & S & \boldsymbol{T} \end{array} \qquad \begin{array}{ccccc} a & b & a & a & \perp \\ \uparrow & \uparrow & \uparrow & \uparrow & \\ S & S & T & \boldsymbol{Z} & \end{array}$$

РЕАЛИЗАЦИЯ ДЕТЕРМИНИРОВАННОГО КОНЕЧНОГО АВТОМАТА

Входная цепочка: массив **C**,

Номер текущего символа: **i**,

Состояние **s: 1, 2, . . . , k, k+1**.

Начальное: **1**, заключительное: **k**, ошибочное: **k+1**.

Матрица переходов: массив **M**.

Таблица перекодировки (символ -> номер столбца): **T**.

```
i:=1; s:=1;
while s<k do begin        {i <= длины строки C}
  j:=T[ord(C[i])];
  s:=M[s,j];
  i:=i+1
end;
if (s=k)and(C[i]='⊥') then «допуск» else «нет»
```

Трудоемкость: $O(n)$

НЕДЕТЕРМИНИРОВАННЫЙ КОНЕЧНЫЙ АВТОМАТ

Грамматика: $G_3(L) = \{\sum, N, S, P\}$

$\sum = \{a, b\}$, $N = \{S, T, C\}$, $S = \{S\}$,

$P = \{S \to aT, S \to aC, T \to aT, T \to aC, T \to bC, C \to bC, C \to b\}$.

Замена: $C \to b$ на $C \to bZ$.

Добавление: $C \to \perp$

Недетерминированности:

$\{S \to aT, S \to aC\}$,

$\{T \to aT, T \to aC\}$,

$\{C \to bC, C \to bZ\}$

Входной символ / Состояние	a	b	$\perp$
S	T/C		
T	T/C	C	
C		C/Z	
Z			*допуск*

Примеры:

a	a	b	b	$\perp$
↑	↑	↑	↑	↑
S	T/C	T/C	C/Z	C/Z

РЕАЛИЗАЦИЯ НЕДЕТЕРМИНИРОВАННОГО КОНЕЧНОГО АВТОМАТА

Входная цепочка: массив **C**, Номер текущего символа: **i**,

Состояние **s**: битовый вектор длиной **k** бит.

Начальное: (1,0,0, . . . 0) заключительное: (?, ?, ?, . . . , ?, 1)

ошибочное: (0,0,0, . . . 0)

Матрица переходов: массив **M** из битовых векторов длиной **k**.

Таблица перекодировки (символ -> номер столбца): **T**.

```
i:=1; s:=(1,0,0, . . . 0);
while (s<>(0,0,0, . . . 0))and(C[i]<>'⊥') do {i<=длины строки C}
begin
  j:=T[ord(C[i])];
  s1:=(0,0,0, . . . 0);
  for p:=1 to k do
    if s[p]=1 then s1:=s1 or M[p,j];
  s:=s1;
  i:=i+1
end;
if (s[k]=1)and(C[i]='⊥') then «допуск» else «нет»
```

Трудоемкость: $O(k^2 n)$ или $O(k n)$

(при использовании битово-векторных операций)

ПРЕОБРАЗОВАНИЕ НЕДЕТЕРМИНИРОВАННОЙ АВТОМАТНОЙ ГРАММАТИКИ К ДЕТЕРМИНИРОВАННОЙ

Пусть в грамматике (с добавленным нетерминалом Z) есть недетерминированность:

$$\{A \to bB_1, A \to bB_2, \ldots, A \to bB_n\}. \qquad (*)$$

Обозначим: $B^*=\{B_1, B_2, \ldots, B_n\}$.

Вместо порождающих правил (*) добавим правило:

$$A \to bB^*$$

В дополнение к правилам $\{B_1 \to \beta_1, B_2 \to \beta_2, \ldots, B_n \to \beta_n\}$ добавим правила: $\{B^* \to \beta_1, B^* \to \beta_2, \ldots, B^* \to \beta_n\}$.

Преобразованная грамматика будет порождать в точности то же самое множество цепочек, что и исходная грамматика, т.е. *язык не изменится*.

Применим такое же преобразование ко всем другим недетерминированностям, в результате грамматика станет *детерминированной*. Общее количество имевшихся в исходной грамматике нетерминалов и вновь добавленных не более чем 2^k, где k – количество имевшихся в исходной грамматике нетерминалов.

Теорема: любую недетерминированную автоматную грамматику можно преобразовать к эквивалентной детерминированной.

Следствие: Все автоматные языки *детерминированные*.

Пример преобразования

Грамматика: $G_3(L) = \{\sum, N, S, P\}$

$\sum = \{a, b\}$, $N = \{S, T, C\}$, $S = \{S\}$,

$P = \{S \to aT, S \to aC, T \to aT, T \to aC, T \to bC, C \to bC, C \to b\}$.

Замена: $C \to b$ на $C \to bZ$, добавление: $Z \to \perp$

Недетерминированности:

$\{S \to aT, S \to aC\}$, $D = \{T, C\}$

$\{T \to aT, T \to aC\}$, то же самое,

$\{C \to bC, C \to bZ\}$, $E = \{C, Z\}$. Здесь E – заключительный нетерминал.

Преобразованные правила:

$$P = \{S \to aD, D \to aD, D \to bE, T \to aD, T \to bC, C \to bE, E \to bE, Z \to \perp, E \to \perp\}$$

Преобразованные правила без лишних правил:

$$P = \{S \to aD, D \to aD, D \to bE, E \to bE, E \to \perp\},$$

Так как нетерминалы T и C не смогут появиться при любом порождении, начиная с начального нетерминала S.

После преобразования в грамматике могут появиться бесполезные порождающие правила и бесполезные нетерминалы. Они не влияют на процесс порождения (и, соответственно, на процесс распознавания), поэтому их можно удалить, не изменяя сам язык.

Алгоритм 1. Обнаружение бесполезных нетерминалов 1 типа (таких, которые не могут быть порождены, начиная с начального нетерминала).

1. Начальный нетерминал помечается как «полезный».
2. Просматриваются все порождающие правила, у которых в левой части имеются нетерминалы, помеченные как «полезные». Нетерминалы в правых частях таких правил также помечаются как «полезные».
3. 2-й шаг повторяется до тех пор, пока не останется непросмотренных правил.
4. Нетерминалы, не помеченные как «полезные», считаются бесполезными.

Алгоритм 2. Обнаружение бесполезных нетерминалов 2 типа (таких, из которых не может быть порождена терминальная цепочка).

1. Просматриваются все порождающие правила, у которых в правой части имеются только терминалы. Нетерминалы в левых частях таких правил помечаются как «полезные».
2. Просматриваются все порождающие правила, у которых в правой части имеются нетерминалы, помеченные как «полезные». Нетерминалы в левых частях таких правил также помечаются как «полезные».
3. 2-й шаг повторяется до тех пор, пока не останется непросмотренных правил.
4. Нетерминалы, не помеченные как «полезные», считаются бесполезными.

После завершения обоих алгоритмов из грамматики необходимо удалить все бесполезные нетерминалы и все порождающие правила, где встречаются эти нетерминалы (как в левой, так и в правой части).

Автоматный язык с несколькими видами цепочек и с символом «⊥»

Дана автоматная грамматика, порождающая цепочки нескольких видов, в конце цепочки имеется символ «⊥». Для каждого вида цепочек предусмотрен отдельный заключительный нетерминал.

Виды цепочек:

- имена (идентификаторы языков программирования);
- целые числа без знака;
- числа с десятичной точкой без знака.

Для сокращения записи вместо отдельных цифр 0,1,. . .,9 будем писать <ц>, а вместо отдельных букв A,B,. . .,Z будем писать <б>. Символ «точка» запишем в кавычках: «.».

Правила грамматики (различные правые части при одинаковой левой части разделены вертикальной чертой):

$\{S \to$ <б>I | <ц>C, $I \to$ <б>I | <ц>I | ⊥, $C \to$ <ц>C | «.»D | ⊥ , $D \to$ <ц>E, $E \to$ <ц>E | ⊥$\}$.

Заключительные нетерминалы: I, C, E.

Конечный автомат с семантическими программами

Кроме перехода в новое состояние в конечном автомате на каждом шаге предусматриваются дополнительные действия (семантические программы). Номера программ указываются в клетках матрицы переходов.

Входной символ / Состояние	<б>	<ц>	«.»	⊥
S	*I* / 1	*C* / 2		
I	*I* / 3	*I* / 3		*имя*
C		*C* / 4	*D* / 5	*цел.число*
D		*E* / 6		
E		*E* / 6		*вещ.число*

Семантические программы

$C[i]$ – текущий входной символ; *name* – символьная строка; n – целочисленная переменная; x, d – вещественные переменные.

1. *name* := $C[i]$;

2. n := ord($C[i]$) – ord('0') ;

3. *name* := *name* + $C[i]$;

4. $n := n * 10 + \text{ord}(C[i]) - \text{ord}('0')$;

5. $d := 1$; $x := n$;

6. $d := d * 0.1$; $x := x + (\text{ord}(C[i]) - \text{ord}('0')) * d$;

Лексический анализатор для языка программирования

Лексемы, выделяемые лексическим анализатором:

- имена (идентификаторы),
- ключевые (служебные) слова,
- целые числа,
- вещественные числа (с десятичной точкой и/или с показателем степени),
- символьные строки,
- составные символы (:= и др.),
- отдельные символы (скобки, знаки операций из одного символа и т.п.).

Лексемы, пропускаемые лексическим анализатором:

- пробелы и символы конца строки,
- комментарии.

Лексический анализатор реализуется как процедура, вызываемая всякий раз, как требуется из входной строки символов выделить очередную лексему.

Пример входа:

C – входная строка символов.

i – номер текущего входного символа в строке *C*, с него начинается анализ; после окончания анализа *i* – номер входного символа *после* лексемы.

Пример выхода:

a – номер распознанной лексемы.

name – символьная строка, содержащая имя, если лексема – имя или строка символов.

n – значение целого числа, если лексема – целое число.

x – значение вещественного числа, если лексема – вещественное число.

Пример. Анализируемая входная строка символов содержит лексемы:

- имена (идентификаторы),
- служебные слова,
- целые числа без знака,
- числа с десятичной точкой без знака,
- символьные строки в одиночных кавычках,
- отдельные символы языка (такие, как +, *, скобки и др.),
- составной символ (:=),
- пробелы, переводы строк (должны пропускаться) ,
- комментарии, начало - символы (/*), конец - символы (*/) (должны пропускаться),
- концевой ограничитель - символ (⊥),
- все другие символы, которые могут встречаться внутри комментария (не символы языка).

Обозначения входных символов: <ц> – цифра, <б> – буква, «.» – точка, « » – пробел, <с> – отдельные символы, ⊥ – концевой ограничитель, <д> – другие символы (не символы языка).

*Z** - *переход в заключительное состояние с действием*: **`i:=i-1`**

ВХОДНОЙ СИМВОЛ / *СОСТОЯНИЕ*	<б>	<ц>	«.»	« »	:	=	/	*	<с>	<д>	⊥
S	*I*	*C*	*Z*	*S*	*A*	*Z*	*K*	*Z*	*Z*		*Z*
I	*I*	*I*	*Z**	*Z*	*Z**	*Z**	*Z**	*Z**	*Z**		*Z**
C		*C*	*D*	*Z*	*Z**	*Z**	*Z**	*Z**	*Z**		*Z**
D		*E*									
E		*E*		*Z*	*Z**	*Z**	*Z**	*Z**	*Z**		*Z**
A	*Z**	*Z**	*Z**	*Z*	*Z**	*Z*	*Z**	*Z**	*Z**		*Z**
K	*Z**	*Z**	*Z**	*Z*	*Z**	*Z**	*Z**	*L*	*Z**		*Z**
L	*L*	*L*	*L*	*L*	*L*	*L*	*L*	*M*	*L*	*L*	
M	*L*	*L*	*L*	*L*	*L*	*L*	*S*	*M*	*L*	*L*	

ПРАВОЛИНЕЙНЫЕ ГРАММАТИКИ

Обобщение автоматных грамматик. Порождающие правила в виде:

$$A \to \omega B \quad \text{или } A \to \omega$$

где A, B – нетерминалы, ω – терминальная цепочка, допустимо: $\omega = \lambda$.

Пример праволинейной грамматики

Грамматика: $G_4(L) = \{\sum, N, S, P\}$

$\sum = \{a, b\}$, $N = \{S, T, C\}$, $S = \{S\}$,

$P = \{S \to aaT,\ T \to aT,\ T \to bbC,\ T \to C,\ C \to a\}$.

Процесс порождения:

$S \Rightarrow aaT \Rightarrow aaC \Rightarrow aaa$,

$S \Rightarrow aaT \Rightarrow aabbC \Rightarrow aabba$,

$S \Rightarrow aaT \Rightarrow aaaT \Rightarrow aaaC \Rightarrow aaaa$,

$S \Rightarrow aaT \Rightarrow aaaT \Rightarrow aaabbC \Rightarrow aabba, \ldots$

Преобразование праволинейной грамматики к автоматной грамматике

1. Пусть в грамматике имеется правило:

$$A \to a_1 a_2 \ldots a_n B, \qquad (*)$$

где $n > 1$.

Добавим в грамматику новые нетерминалы:

$$B_1, B_2, \ldots, B_{n-1}$$

И вместо правила (*) добавим правила:

$$A \to a_1 B_1,\ B_1 \to a_2 B_2,\ \ldots,\ B_{n-1} \to a_n B$$

2. Пусть в грамматике имеется правило:

$$A \to B, \qquad (**)$$

а также правила: $B \to \beta_1$, $B \to \beta_2$, . . . , $B \to \beta_m$,

Вместо правила (**) добавим правила:

$$A \to \beta_1, A \to \beta_2, \ldots, A \to \beta_m$$

3. Пусть в грамматике имеется правило:

$$A \to \lambda, \qquad (***)$$

а также правила: $B_1 \to \omega_1 A$, $B_2 \to \omega_2 A$, . . . , $B_k \to \omega_k A$,

Вместо правила (***) добавим правила:

$$B_1 \to \omega_1, B_2 \to \omega_2, \ldots, B_k \to \omega_k,$$

4. Пусть в грамматике имеется правило:

$$S \to \lambda, \qquad (****)$$

где S – начальный нетерминал.

Это порождающее правило оставляем без изменения.

В результате преобразований язык *не изменится*.

Трудоёмкость преобразования определяется количеством новых нетерминалов.

Если в языке не было правила $S \to \lambda$ и оно не появилось после всех преобразований, то получившаяся грамматика будет автоматной.

Теорема.

Любая праволинейная грамматика эквивалентна автоматной грамматике, к которой (возможно) добавлено порождающее правило:

$$S \to \lambda.$$

Следствие: множество праволинейных языков ***почти*** совпадает с множеством автоматных языков.

Пример преобразования праволинейной грамматики

Грамматика: $G_4(L) = \{\sum, N, S, P\}$,

$\sum = \{a, b\}$, $N = \{S, T, C\}$, $S = \{S\}$,

$P = \{S \to aaT,\ T \to aT, T \to bbC, T \to C, C \to a\}$.

Автоматная грамматика: $G_5(L) = \{\sum, N, S, P\}$,

$\sum = \{a, b\}$, $N = \{S, T, A, B, C\}$, $S = \{S\}$,

$P = \{S \to aA, A \to aT, T \to aT, T \to bB, T \to a,$

$B \to bC, C \to a\}$.

Регулярные множества

Регулярные множества – это следующие множества цепочек символов из некоторого алфавита Σ:

- Пустое множество $\varnothing$.
- Множество из пустой цепочки $\{\lambda\}$.
- Множество из любого символа $\{a\}$ алфавита Σ.
- Множество всех возможных цепочек вида $\alpha\beta$ (конкатенация); $\alpha \in A$, $\beta \in B$, где A, B – регулярные множества.
- Объединение множеств $A \cup B$, где A, B – регулярные множества.
- Объединение множеств всех возможных цепочек вида λ, α_1, $\alpha_1\alpha_2$, $\alpha_1\alpha_2\alpha_3$, . . . и т.д., где все α_1, α_2, α_3, . . . принадлежат регулярному множеству A (A^* – транзитивное замыкание A).

Регулярные выражения

Регулярное выражение – это шаблон для задания регулярного множества цепочек символов из некоторого алфавита Σ.

Кроме символов алфавита Σ в регулярное выражение могут входить вспомогательные метасимволы: Ø (пустое множество), λ (пустая строка), скобки { }, скобки { }*, вертикальная черта |.

- Пустое множество обозначается знаком Ø.
- Пустая цепочка обозначается знаком λ.
- Символ алфавита Σ обозначает себя сам.
- Если α и β – оба регулярные выражения, то запись вида αβ – регулярное выражение, конкатенация цепочек из α и β ;
- Если α и β – оба регулярные выражения, то запись вида α | β – регулярное выражение, обозначающее объединение множеств цепочек из α и β ;
- Если α – регулярное выражения, то запись вида {α} – то же самое регулярное выражение, рассматриваемое, как единое целое;
- Если α – регулярное выражения, то запись вида {α}* – регулярное выражение, обозначающее объединение множеств цепочек из:

  λ, α, αα, ααα, . . .

  и т.д., ({α}* – транзитивное замыкание α).

Примеры

1. Регулярное выражение:

$$\{\lambda|+|-\}\{0|1|2|3|4|5|6|7|8|9\}\{0|1|2|3|4|5|6|7|8|9\}^*$$

задает запись целого числа без знака или со знаком «+» или «–».

Для краткости вместо явного перечисления цифр или букв через символ «|», будем использовать многоточие.

---

2. Регулярное выражение:

$$\{0|1|...|9\}\{0|1|...|9\}^* .\{0|1|...|9\}\{0|1|...|9\}^*$$

задает запись беззнакового десятичного числа с дробной частью.

---

3. Регулярное выражение:

$$\{A|B|...|Z\}\{0|1|...|9|A|B|...|Z\}^*$$

задает запись идентификатора (имени) для языка программирования.

Преобразование регулярного выражения к праволинейной грамматике

1. Пусть задано регулярное выражение α. Запишем прообраз порождающего правила в виде: $S \to \alpha$, где S – начальный нетерминал.
2. Пусть прообраз порождающего правила имеет вид: $A \to a\beta$,

   где A – некоторый нетерминал, a – терминал. Заменим это правило на следующие: $A \to aB$, $B \to \beta$, где B – новый нетерминал.
3. Пусть прообраз порождающего правила имеет вид:

   $A \to \{\alpha_1|\alpha_2\}\beta$, где A – некоторый нетерминал. Заменим это правило на следующие: $A \to \alpha_1\beta$, $A \to \alpha_2\beta$.
4. Пусть прообраз порождающего правила имеет вид:

   $A \to \{\alpha\}^*\beta$, где A – некоторый нетерминал. Заменим это правило на два прообраза: 1) $A \to \beta$, 2) $A \to \alpha A$.

Преобразования проводятся до тех пор, пока все полученные порождающие правила не станут праволинейными.

Регулярные множества есть множество автоматных языков!

Примеры преобразования

Регулярное выражение:

$$\{\lambda|+|-\}\{0|1|\ldots|9\}\{0|1|\ldots|9\}^*$$

преобразуется в праволинейную грамматику с правилами:

$$\{S \to T,\ S \to -T,\ S \to +T,\ T \to \langle ц\rangle C,\ C \to \langle ц\rangle C,\ C \to \lambda\}.$$

Регулярное выражение:

$$\{0|1|\ldots|9\}\{0|1|\ldots|9\}^*.\{0|1|\ldots|9\}\{0|1|\ldots|9\}^*$$

преобразуется в праволинейную грамматику с правилами:

$$\{S \to \langle ц\rangle T,\ T \to \langle ц\rangle T,\ T \to «.»C,\ C \to \langle ц\rangle D,\ D \to \lambda,\ D \to \langle ц\rangle D\}.$$

Регулярное выражение:

$$\{A|B|\ldots|Z\}\{0|1|\ldots|9|A|B|\ldots|Z\}^*$$

преобразуется в праволинейную грамматику с правилами:

$$\{S \to \langle б\rangle I,\ I \to \langle б\rangle I,\ I \to \langle ц\rangle I,\ I \to \lambda\}.$$